
Indian Institute of Technology Madras Zanzibar

School of Engineering and Science

Z2004: Comprehensive SQL Reference

T-SQL (SQL Server) and PostgreSQL — Every Command, Every Clause

Z2004 — Database Management Systems

Document 4 of 6 | B.S. Data Science and Artificial Intelligence, Year 2

Even Semester: March – July 2026

Dr. Innocent Nyalala

Assistant Professor of Data Science and AI

Head, SAAIL Lab | IIT Madras Zanzibar

✉ innocent@iitmz.ac.in | 🕒 Office Hours: Wednesday 2:00–4:00 PM | 📍 Room 106

Teaching Assistants

Support hours: **Friday 2:00–4:00 PM (EAT)**

Sri Advaita Varakavi | ✉ zda23b010@iitmz.ac.in

Saleh Saleh | ✉ zda23b001@iitmz.ac.in

Primary (Weeks 9+): T-SQL on Microsoft SQL Server 2022

PostgreSQL differences annotated as [PG: ...]

Contents

1 SQL Categories and Execution Order	2
2 Data Types	2
3 Reference Schema (Used Throughout This Reference)	2
4 DDL — Data Definition Language	5
5 DML — SELECT Queries	6
6 DML — INSERT, UPDATE, DELETE, MERGE	7
7 DML — Joins	8
8 DML — Subqueries and CTEs	9
9 Set Operators (UNION, INTERSECT, EXCEPT)	10
10 Expressions, NULL Handling, and Type Conversion	11
11 Useful Built-in Functions (Quick Sheet)	11
12 Grouping Extensions and Pagination	12
13 Temporary Tables and Common Utilities	12
14 Query Plans and Performance (Basics)	13
15 JSON and Full-Text Search (Optional but Common in Modern SQL)	13
16 Window Functions	13
17 Indexes	14
18 Views	15
19 Stored Procedures and Functions	15
20 Triggers	17
21 Transactions and Concurrency	17
22 Security — DCL	18
23 Vector Database: pgvector (PostgreSQL)	19
24 Python Integration	19

1. SQL Categories and Execution Order

📖 The Five SQL Categories

DDL	CREATE ALTER DROP TRUNCATE
DML	SELECT INSERT UPDATE DELETE MERGE
DCL	GRANT REVOKE
TCL	BEGIN COMMIT ROLLBACK SAVEPOINT
DQL	SELECT (sometimes treated separately)

📖 SELECT Logical Execution Order

1. FROM + JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY
8. TOP / LIMIT / FETCH

You write SQL in a different order than the engine executes it.

2. Data Types

📖 T-SQL Data Types (SQL Server)

Integer	INT BIGINT SMALLINT TINYINT
Decimal	DECIMAL(p,s) NUMERIC(p,s) MONEY
Float	FLOAT REAL
String	VARCHAR(n) NVARCHAR(n) CHAR(n)
Text	VARCHAR(MAX) NVARCHAR(MAX)
Date/Time	DATE TIME DATETIME DATETIME2
Boolean	BIT (0 = false, 1 = true)
Binary	VARBINARY(n) VARBINARY(MAX)
Other	UNIQUEIDENTIFIER XML ROWVERSION

📖 PostgreSQL Data Types

Integer	INTEGER BIGINT SMALLINT SERIAL
Decimal	DECIMAL(p,s) NUMERIC(p,s)
Float	DOUBLE PRECISION REAL
String	VARCHAR(n) CHAR(n) TEXT
Date/Time	DATE TIME TIMESTAMP TIMESTAMPZ
Boolean	BOOLEAN (TRUE/FALSE)
JSON	JSON JSONB
Array	INTEGER[] TEXT[]
Vector	vector(n) (via pgvector extension)
UUID	UUID

3. Reference Schema (Used Throughout This Reference)

Tables and Key Relationships (PK/FK)

Departments	DeptID (PK)
Students	StudentID (PK), DeptID (FK → Departments.DeptID)
Courses	CourseID (PK)
Enrollments	(StudentID, CourseID) (PK), FKs to Students + Courses
Orders	OrderID (PK) (used for window/running-total examples)
Accounts	AccountID (PK) (used for transactions examples)
Employees	EmployeeID (PK), ManagerID (self-FK) (used for self-join/recursive CTE)
DeptTransfers	StudentID (PK, FK), NewDeptID (FK) (used for UPDATE...JOIN)
Graduates	StudentID (PK, FK) (used for INSERT...SELECT)
Suspended	StudentID (PK, FK) (used for DELETE...IN subquery)
NewEnrollments	staging table used for MERGE / upsert examples
AuditLog	LogID (PK) (used for trigger logging)
Documents	DocID (PK) (used for JSON, full-text, and pgvector examples)

Schema DDL (SQL Server 2022 primary; PG notes inline)

```

1  -- =====
2  -- Core master tables
3  -- =====
4  CREATE TABLE Departments (
5      DeptID      INT          PRIMARY KEY,
6      DeptName    NVARCHAR(100) NOT NULL,
7      Location    NVARCHAR(100) NULL
8  );
9
10 CREATE TABLE Students (
11     StudentID    INT          IDENTITY(1,1) PRIMARY KEY,      -- [PG]:
12                     use GENERATED ALWAYS AS IDENTITY
13     FullName     NVARCHAR(100) NOT NULL,
14     Email        NVARCHAR(150) NOT NULL UNIQUE,
15     PhoneNumber  NVARCHAR(20)  NULL,
16     DOB          DATE          NOT NULL,
17     GPA          DECIMAL(3,2)  NOT NULL CHECK (GPA BETWEEN 0 AND 4),
18     DeptID       INT          NULL,
19     ProfileJson  NVARCHAR(MAX) NULL,                          -- [PG]:
20                     use JSONB
21     CreatedAt    DATETIME2     NOT NULL DEFAULT GETDATE(),   -- [PG]:
22                     DEFAULT NOW()
23     CONSTRAINT fk_students_dept
24         FOREIGN KEY (DeptID) REFERENCES Departments (DeptID)
25         ON DELETE SET NULL ON UPDATE CASCADE
26 );
27
28 CREATE TABLE Courses (
29     CourseID     INT          PRIMARY KEY,
30     CourseName   NVARCHAR(120) NOT NULL
31 );
32
33 CREATE TABLE Enrollments (
34     StudentID    INT NOT NULL,
35     CourseID     INT NOT NULL,
36     EnrolledAt   DATETIME2 NOT NULL DEFAULT GETDATE(),
37     CONSTRAINT pk_enrollments PRIMARY KEY (StudentID, CourseID),
38     CONSTRAINT fk_enr_student FOREIGN KEY (StudentID) REFERENCES Students
39         (StudentID) ON DELETE CASCADE,
40     CONSTRAINT fk_enr_course  FOREIGN KEY (CourseID)  REFERENCES Courses(
41         CourseID) ON DELETE CASCADE
42 );

```

```

38
39 -- =====
40 -- Tables used in specific examples
41 -- =====
42 CREATE TABLE Orders (
43     OrderID     INT IDENTITY(1,1) PRIMARY KEY,
44     OrderDate   DATE             NOT NULL,
45     Amount      DECIMAL(12,2) NOT NULL CHECK (Amount >= 0)
46 );
47
48 CREATE TABLE Accounts (
49     AccountID INT PRIMARY KEY,
50     Balance   DECIMAL(12,2) NOT NULL
51 );
52
53 CREATE TABLE Employees (
54     EmployeeID INT PRIMARY KEY,
55     Name        NVARCHAR(100) NOT NULL,
56     ManagerID   INT NULL,
57     CONSTRAINT fk_emp_manager
58         FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)
59 );
60
61 CREATE TABLE DeptTransfers (
62     StudentID INT PRIMARY KEY,
63     NewDeptID INT NOT NULL,
64     CONSTRAINT fk_transfer_student FOREIGN KEY (StudentID) REFERENCES
65         Students(StudentID) ON DELETE CASCADE,
66     CONSTRAINT fk_transfer_dept    FOREIGN KEY (NewDeptID) REFERENCES
67         Departments(DeptID)
68 );
69
70 CREATE TABLE Graduates (
71     StudentID INT PRIMARY KEY,
72     FullName  NVARCHAR(100) NOT NULL,
73     GPA        DECIMAL(3,2) NOT NULL,
74     CONSTRAINT fk_grads_student FOREIGN KEY (StudentID) REFERENCES
75         Students(StudentID)
76 );
77
78 CREATE TABLE Suspended (
79     StudentID INT PRIMARY KEY,
80     Reason     NVARCHAR(200) NULL,
81     CONSTRAINT fk_susp_student FOREIGN KEY (StudentID) REFERENCES
82         Students(StudentID) ON DELETE CASCADE
83 );
84
85 CREATE TABLE NewEnrollments (
86     StudentID INT PRIMARY KEY,
87     FullName  NVARCHAR(100) NOT NULL,
88     Email     NVARCHAR(150) NOT NULL,
89     GPA        DECIMAL(3,2) NOT NULL CHECK (GPA BETWEEN 0 AND 4),
90     DeptID     INT NULL
91 );
92
93 CREATE TABLE AuditLog (
94     LogID      INT IDENTITY(1,1) PRIMARY KEY,
95     Action     NVARCHAR(20) NOT NULL,
96     TableName  NVARCHAR(128) NOT NULL,
97     ActionTime DATETIME2 NOT NULL DEFAULT GETDATE()
98 );
99

```

```

96 CREATE TABLE Documents (
97     DocID          INT IDENTITY(1,1) PRIMARY KEY,
98     Content        NVARCHAR(MAX) NOT NULL,
99     MetadataJson   NVARCHAR(MAX) NULL   -- [PG]: use JSONB
100 );
101
102 -- [PG/pgvector section]
103 -- CREATE EXTENSION IF NOT EXISTS vector;
104 -- ALTER TABLE Documents ADD COLUMN Embedding vector(384);

```

4. DDL — Data Definition Language

CREATE TABLE (Pattern) (See Reference Schema for the full course schema)

```

1  -- Template: keys, constraints, identity/defaults
2  CREATE TABLE Example (
3      ID          INT IDENTITY(1,1) PRIMARY KEY,      -- [PG]: GENERATED
4      ALWAYS AS IDENTITY
5      Name        NVARCHAR(100) NOT NULL,
6      Email       NVARCHAR(150) NOT NULL UNIQUE,
7      Score       DECIMAL(5,2) NOT NULL CHECK (Score >= 0),
8      DeptID      INT NULL,
9      CreatedAt   DATETIME2 NOT NULL DEFAULT GETDATE(), -- [PG]: DEFAULT
10     NOW()
11     CONSTRAINT fk_example_dept
12     FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
13     ON DELETE SET NULL ON UPDATE CASCADE
14 );

```

ALTER TABLE

```

1  -- Add / drop column
2  ALTER TABLE Students ADD MiddleName NVARCHAR(50) NULL;
3  ALTER TABLE Students DROP COLUMN MiddleName;
4
5  -- Alter column type
6  ALTER TABLE Students ALTER COLUMN GPA DECIMAL(4,3); -- T-SQL
7  -- [PG]: ALTER TABLE Students ALTER COLUMN GPA TYPE DECIMAL(4,3);
8
9  -- Add / drop constraint
10 ALTER TABLE Students ADD CONSTRAINT chk_gpa CHECK (GPA >= 0);
11 ALTER TABLE Students DROP CONSTRAINT chk_gpa;
12
13 -- Add foreign key
14 ALTER TABLE Students ADD FOREIGN KEY (DeptID) REFERENCES Departments(
    DeptID);

```

DROP, TRUNCATE, and RENAME

```

1  DROP TABLE Students;           -- Deletes table AND all data
   permanently
2  DROP TABLE IF EXISTS Students; -- Safe drop (no error if not exists)

```

```

3 TRUNCATE TABLE Students;           -- Deletes all rows; keeps structure
  ; faster than DELETE
4 EXEC sp_rename 'Students', 'Learners'; -- T-SQL: rename table
5 -- [PG]: ALTER TABLE Students RENAME TO Learners;

```

</> Constraints Reference

```

1 -- Column-level constraints
2 ColName INT NOT NULL,
3 ColName INT UNIQUE,
4 ColName INT PRIMARY KEY,
5 ColName INT DEFAULT 0,
6 ColName INT CHECK (ColName > 0),
7 ColName INT REFERENCES OtherTable(OtherCol),
8
9 -- Table-level constraints (after all columns)
10 CONSTRAINT pk_name PRIMARY KEY (Col1, Col2), -- composite PK
11 CONSTRAINT uq_name UNIQUE (Col1, Col2),
12 CONSTRAINT fk_name FOREIGN KEY (Col1) REFERENCES T2(C1)
13 ON DELETE CASCADE ON UPDATE SET NULL,
14 CONSTRAINT chk_name CHECK (Col1 < Col2)

```

5. DML — SELECT Queries

</> SELECT Fundamentals

```

1 SELECT * FROM Students;
2 SELECT StudentID, FullName, GPA FROM Students;
3 SELECT TOP 10 * FROM Students ORDER BY GPA DESC; -- T-SQL
4 -- [PG]: SELECT * FROM Students ORDER BY GPA DESC LIMIT 10;
5 SELECT DISTINCT DeptID FROM Students;
6 SELECT FullName AS Name, GPA * 4 AS ScaledGPA FROM Students;
7 SELECT * FROM Students WHERE GPA > 3.5 AND DeptID = 2;
8 SELECT * FROM Students WHERE FullName LIKE 'A%'; -- starts
  with A
9 SELECT * FROM Students WHERE GPA BETWEEN 3.0 AND 3.9;
10 SELECT * FROM Students WHERE DeptID IN (1, 3, 5);
11 SELECT * FROM Students WHERE DeptID IS NULL;
12 SELECT * FROM Students WHERE DeptID IS NOT NULL;
13 SELECT * FROM Students ORDER BY GPA DESC, FullName ASC;

```

</> Aggregate Functions

```

1 SELECT COUNT(*) AS TotalStudents FROM Students;
2 SELECT COUNT(DeptID) AS WithDept FROM Students; -- ignores
  NULLs
3 SELECT COUNT(DISTINCT DeptID) AS NumDepts FROM Students;
4 SELECT SUM(GPA) AS TotalGPA FROM Students;
5 SELECT AVG(GPA) AS AvgGPA FROM Students;
6 SELECT MAX(GPA), MIN(GPA) FROM Students;
7 SELECT DeptID,
8 COUNT(*) AS NumStudents,
9 AVG(GPA) AS AvgGPA,

```

```

10      MAX(GPA)                                AS TopGPA
11 FROM    Students
12 GROUP BY DeptID
13 HAVING  AVG(GPA) > 3.0
14 ORDER BY AvgGPA DESC;

```

6. DML — INSERT, UPDATE, DELETE, MERGE

</> INSERT

```

1  -- Single row
2  INSERT INTO Students (FullName, Email, DOB, GPA, DeptID)
3  VALUES ('Amina Hassan', 'amina@iitmz.ac.tz', '2004-03-15', 3.85, 2);
4
5  -- Multiple rows
6  INSERT INTO Students (FullName, Email, DOB, GPA, DeptID) VALUES
7      ('John Mwanga', 'john@iitmz.ac.tz', '2003-07-22', 3.60, 1),
8      ('Fatuma Ali', 'fatuma@iitmz.ac.tz', '2004-01-10', 3.92, 3);
9
10 -- Insert from SELECT
11 INSERT INTO Graduates (StudentID, FullName, GPA)
12 SELECT StudentID, FullName, GPA FROM Students WHERE GPA >= 3.5;

```

</> UPDATE and DELETE

```

1  -- UPDATE with WHERE (always use WHERE!)
2  UPDATE Students SET GPA = 3.95 WHERE StudentID = 101;
3  UPDATE Students SET GPA = GPA + 0.1 WHERE DeptID = 2 AND GPA < 3.8;
4
5  -- UPDATE with JOIN (T-SQL)
6  -- Example: apply a transfer table to move students to a new department
7  UPDATE s
8  SET   s.DeptID = t.NewDeptID
9  FROM   Students s
10 JOIN   DeptTransfers t ON t.StudentID = s.StudentID;
11
12 -- DELETE with WHERE (always use WHERE!)
13 DELETE FROM Students WHERE GPA < 1.0;
14 DELETE FROM Students WHERE StudentID IN (SELECT StudentID FROM Suspended)
15 ;
16 -- TRUNCATE (removes ALL rows, no rollback in most cases)
17 TRUNCATE TABLE Orders;

```

</> MERGE (Upsert)

```

1  -- T-SQL MERGE (upsert pattern)
2  MERGE INTO Students AS target
3  USING NewEnrollments AS source ON target.StudentID = source.StudentID
4  WHEN MATCHED THEN
5      UPDATE SET target.FullName = source.FullName,
6                target.Email    = source.Email,
7                target.GPA      = source.GPA,

```

```

8         target.DeptID    = source.DeptID
9 WHEN NOT MATCHED BY TARGET THEN
10     INSERT (StudentID, FullName, Email, GPA, DeptID)
11     VALUES (source.StudentID, source.FullName, source.Email, source.GPA,
12             source.DeptID);
13
14 -- [PG]: Use INSERT ... ON CONFLICT (equivalent upsert)
15 INSERT INTO Students (StudentID, FullName, Email, GPA, DeptID)
16 VALUES (101, 'Updated Name', 'updated@iitmz.ac.tz', 3.90, 2)
17 ON CONFLICT (StudentID)
18 DO UPDATE SET FullName = EXCLUDED.FullName,
19              Email     = EXCLUDED.Email,
20              GPA       = EXCLUDED.GPA,
21              DeptID    = EXCLUDED.DeptID;

```

7. DML — Joins

⌘ All JOIN Types

```

1 -- INNER JOIN: only matching rows
2 SELECT s.FullName, d.DeptName
3 FROM   Students s INNER JOIN Departments d ON s.DeptID = d.DeptID;
4
5 -- LEFT OUTER JOIN: all students, even those with no department
6 SELECT s.FullName, d.DeptName
7 FROM   Students s LEFT JOIN Departments d ON s.DeptID = d.DeptID;
8
9 -- RIGHT OUTER JOIN: all departments, even empty ones
10 SELECT s.FullName, d.DeptName
11 FROM   Students s RIGHT JOIN Departments d ON s.DeptID = d.DeptID;
12
13 -- FULL OUTER JOIN: all rows from both sides
14 SELECT s.FullName, d.DeptName
15 FROM   Students s FULL OUTER JOIN Departments d ON s.DeptID = d.DeptID;
16
17 -- CROSS JOIN: every student paired with every course
18 SELECT s.FullName, c.CourseName
19 FROM   Students s CROSS JOIN Courses c;
20
21 -- SELF JOIN: employee and their manager (same table)
22 SELECT e.Name AS Employee, m.Name AS Manager
23 FROM   Employees e LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;

```

⌘ JOIN Patterns (Semi, Anti, APPLY/LATERAL)

```

1 -- SEMI JOIN: rows in A that have a match in B
2 SELECT s.*
3 FROM   Students s
4 WHERE  EXISTS (SELECT 1 FROM Enrollments e WHERE e.StudentID = s.
5             StudentID);
6
7 -- ANTI JOIN: rows in A that do NOT have a match in B
8 SELECT s.*
9 FROM   Students s
10 WHERE NOT EXISTS (SELECT 1 FROM Enrollments e WHERE e.StudentID = s.

```

```

StudentID);
10
11 -- Join and filter: filter in ON (keeps LEFT JOIN rows) vs WHERE (can
    turn it into INNER JOIN)
12 SELECT s.FullName, e.CourseID
13 FROM   Students s
14 LEFT   JOIN Enrollments e
15         ON e.StudentID = s.StudentID
16         AND e.CourseID = 101; -- filter on joined table but preserve non
    -matches
17
18 -- APPLY / LATERAL (row-by-row derived table)
19 -- T-SQL: CROSS APPLY / OUTER APPLY
20 SELECT s.StudentID, x.TopCourseID
21 FROM   Students s
22 CROSS  APPLY (
23     SELECT TOP 1 e.CourseID AS TopCourseID
24     FROM   Enrollments e
25     WHERE  e.StudentID = s.StudentID
26     ORDER  BY e.EnrolledAt DESC
27 ) x;
28
29 -- [PG]: use LATERAL instead of APPLY
30 -- SELECT s.studentid, x.topcourseid
31 -- FROM students s
32 -- JOIN LATERAL (
33 --     SELECT e.courseid AS topcourseid
34 --     FROM enrollments e
35 --     WHERE e.studentid = s.studentid
36 --     ORDER BY e.enrolledat DESC
37 --     LIMIT 1
38 -- ) x ON TRUE;

```

8. DML — Subqueries and CTEs

Subquery Types

```

1 -- Scalar subquery (returns one value)
2 SELECT FullName FROM Students
3 WHERE GPA = (SELECT MAX(GPA) FROM Students);
4
5 -- Table subquery with IN
6 SELECT FullName FROM Students
7 WHERE DeptID IN (SELECT DeptID FROM Departments WHERE Location = '
    Zanzibar');
8
9 -- EXISTS (correlated)
10 SELECT d.DeptName FROM Departments d
11 WHERE EXISTS (
12     SELECT 1 FROM Students s WHERE s.DeptID = d.DeptID AND s.GPA > 3.8
13 );
14
15 -- NOT EXISTS
16 SELECT d.DeptName FROM Departments d
17 WHERE NOT EXISTS (SELECT 1 FROM Students s WHERE s.DeptID = d.DeptID);
18
19 -- Correlated subquery (re-runs for each outer row)
20 SELECT s1.FullName, s1.GPA

```

```

21 FROM Students s1
22 WHERE s1.GPA > (SELECT AVG(s2.GPA) FROM Students s2 WHERE s2.DeptID = s1.
    DeptID);

```

Common Table Expressions (WITH)

```

1  -- Simple CTE
2  WITH HighPerformers AS (
3      SELECT StudentID, FullName, GPA
4      FROM Students
5      WHERE GPA > 3.7
6  )
7  SELECT * FROM HighPerformers ORDER BY GPA DESC;
8
9  -- Multiple CTEs
10 WITH DeptStats AS (
11     SELECT DeptID, AVG(GPA) AS AvgGPA, COUNT(*) AS NumStudents
12     FROM Students GROUP BY DeptID
13 ),
14 TopDepts AS (
15     SELECT DeptID FROM DeptStats WHERE AvgGPA > 3.5
16 )
17 SELECT d.DeptName, ds.AvgGPA
18 FROM Departments d
19 JOIN DeptStats ds ON d.DeptID = ds.DeptID
20 JOIN TopDepts td ON d.DeptID = td.DeptID;
21
22 -- Recursive CTE (org chart)
23 WITH OrgChart AS (
24     SELECT EmployeeID, Name, ManagerID, 0 AS Level
25     FROM Employees WHERE ManagerID IS NULL -- anchor
26     UNION ALL
27     SELECT e.EmployeeID, e.Name, e.ManagerID, oc.Level + 1
28     FROM Employees e JOIN OrgChart oc ON e.ManagerID = oc.EmployeeID
29     -- recursive
30 )
31 SELECT * FROM OrgChart ORDER BY Level, Name;

```

9. Set Operators (UNION, INTERSECT, EXCEPT)

UNION / INTERSECT / EXCEPT

```

1  -- UNION: combine rows, remove duplicates
2  SELECT DeptID FROM Students
3  UNION
4  SELECT DeptID FROM Departments;
5
6  -- UNION ALL: keep duplicates (faster)
7  SELECT DeptID FROM Students
8  UNION ALL
9  SELECT DeptID FROM Departments;
10
11 -- INTERSECT: common rows
12 SELECT DeptID FROM Students
13 INTERSECT

```

```

14 SELECT DeptID FROM Departments;
15
16 -- EXCEPT: rows in first query not in second
17 SELECT DeptID FROM Students
18 EXCEPT
19 SELECT DeptID FROM Departments;
20
21 -- Notes: all SELECTs must have same number of columns and compatible
    types

```

10. Expressions, NULL Handling, and Type Conversion

⌘ CASE, COALESCE, NULLIF, CAST

```

1  -- CASE (conditional)
2  SELECT FullName,
3         CASE WHEN GPA >= 3.7 THEN 'A'
4              WHEN GPA >= 3.0 THEN 'B'
5              ELSE 'C/D' END AS Grade
6  FROM Students;
7
8  -- NULL handling
9  SELECT COALESCE(PhoneNumber, 'N/A') AS Phone FROM Students; -- first non-
    NULL
10 SELECT NULLIF(GPA, 0) AS GPA_or_NULL FROM Students;          -- returns
    NULL if GPA=0
11 SELECT ISNULL(PhoneNumber, 'N/A') AS Phone FROM Students;    -- T-SQL only
12 -- [PG]: use COALESCE(...) (no ISNULL)
13
14 -- Type conversion
15 SELECT CAST(GPA AS INT) AS GPATrunc FROM Students;
16 SELECT CONVERT(VARCHAR(10), DOB, 23) AS DOB_ISO FROM Students; -- T-SQL
17 -- [PG]: SELECT TO_CHAR(DOB, 'YYYY-MM-DD') AS dob_iso FROM Students;

```

11. Useful Built-in Functions (Quick Sheet)

⌘ String + Date/Time + Numeric

```

1  -- Strings (T-SQL / PG mostly similar)
2  SELECT UPPER(FullName), LOWER(FullName) FROM Students;
3  SELECT SUBSTRING(FullName, 1, 5) FROM Students;          -- T-SQL
4  -- [PG]: SELECT SUBSTRING(FullName FROM 1 FOR 5) FROM Students;
5  SELECT REPLACE(FullName, ' ', '-') FROM Students;
6  SELECT TRIM(' abc ') AS Trimmed;
7  SELECT CONCAT(FullName, ' <', Email, '>') FROM Students;
8
9  -- Dates
10 SELECT GETDATE() AS Now;                                   -- T-SQL
11 -- [PG]: SELECT NOW() AS now;
12 SELECT DATEADD(day, 7, GETDATE()) AS Plus7Days;          -- T-SQL
13 -- [PG]: SELECT NOW() + INTERVAL '7 days' AS plus7days;
14 SELECT DATEDIFF(day, DOB, GETDATE()) AS DaysAlive;       -- T-SQL
15 -- [PG]: SELECT (CURRENT_DATE - DOB) AS daysalive FROM Students;

```

```

16
17 -- Numeric
18 SELECT ROUND(GPA, 1), CEILING(GPA), FLOOR(GPA), ABS(GPA) FROM Students;

```

12. Grouping Extensions and Pagination

⌘ ROLLUP / CUBE / GROUPING SETS + OFFSET/FETCH

```

1  -- GROUPING SETS / ROLLUP / CUBE (supported in SQL Server and PostgreSQL)
2  SELECT DeptID, COUNT(*) AS NumStudents
3  FROM    Students
4  GROUP BY GROUPING SETS ((DeptID), ()); -- per-dept + grand total
5
6  SELECT DeptID, COUNT(*) AS NumStudents
7  FROM    Students
8  GROUP BY ROLLUP (DeptID);              -- per-dept + grand total
9
10 -- Pagination
11 SELECT *
12 FROM    Students
13 ORDER BY StudentID
14 OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY; -- T-SQL + PG
15
16 -- [PG]: also common: LIMIT 10 OFFSET 20;

```

13. Temporary Tables and Common Utilities

⌘ Temp Tables, Variables, Sequences/Identity

```

1  -- T-SQL temp tables / table variables
2  CREATE TABLE #TempTop (StudentID INT, GPA DECIMAL(3,2));
3  INSERT INTO #TempTop SELECT TOP 10 StudentID, GPA FROM Students ORDER BY
4  GPA DESC;
5
6  DECLARE @Top TABLE (StudentID INT, GPA DECIMAL(3,2));
7  INSERT INTO @Top SELECT TOP 10 StudentID, GPA FROM Students ORDER BY GPA
8  DESC;
9
10 -- [PG]: temporary table
11 -- CREATE TEMP TABLE temp_top(studentid INT, gpa NUMERIC(3,2));
12 -- INSERT INTO temp_top SELECT studentid, gpa FROM students ORDER BY gpa
13 -- DESC LIMIT 10;
14
15 -- Identity / sequences
16 SELECT SCOPE_IDENTITY() AS LastID; -- T-SQL
17
18 -- [PG]: SELECT currval(pg_get_serial_sequence('students','studentid'));

```

14. Query Plans and Performance (Basics)

</> EXPLAIN, Statistics, and Sargability

```

1  -- PostgreSQL: see plan and actual timing
2  EXPLAIN ANALYZE
3  SELECT * FROM Students WHERE DeptID = 2 AND GPA >= 3.5;
4
5  -- SQL Server: (in SSMS) include actual execution plan +
6  SET STATISTICS IO ON;
7  SET STATISTICS TIME ON;
8  SELECT * FROM Students WHERE DeptID = 2 AND GPA >= 3.5;
9
10 -- Sargable predicate: good (uses index)
11 SELECT * FROM Students WHERE DOB >= '2004-01-01';
12
13 -- Non-sargable predicate: often bad (prevents index seek)
14 SELECT * FROM Students WHERE YEAR(DOB) = 2004; -- T-SQL
15 -- [PG]: EXTRACT(YEAR FROM dob) = 2004 has similar issue

```

15. JSON and Full-Text Search (Optional but Common in Modern SQL)**</> JSON (T-SQL vs PG) + Full-Text**

```

1  -- T-SQL JSON (stored as NVARCHAR)
2  SELECT JSON_VALUE(ProfileJson, '$.city') AS City FROM Students;
3  SELECT JSON_VALUE(MetadataJson, '$.source') AS Source FROM Documents;
4
5  -- [PG] JSONB operators (if columns are JSONB)
6  -- SELECT profilejson ->> 'city' AS city FROM students;
7  -- SELECT * FROM students WHERE profilejson @> '{"city": "Zanzibar"}';
8  -- SELECT metadatajson ->> 'source' AS source FROM documents;
9
10 -- Full-text search
11 -- T-SQL (requires full-text index): CONTAINS/CONTAINSTABLE
12 -- SELECT * FROM Documents WHERE CONTAINS(Content, '"database"');
13
14 -- [PG] full-text search
15 -- SELECT * FROM documents WHERE to_tsvector('english', content) @@
    plainto_tsquery('english', 'database');

```

16. Window Functions**</> Window Function Syntax and All Types**

```

1  -- Syntax template
2  function_name() OVER (
3      [PARTITION BY col1, col2] -- divide into groups (optional)
4      [ORDER BY col3 DESC]      -- ordering within each partition
5      [ROWS|RANGE BETWEEN ...] -- frame specification (optional)
6  )
7

```

```

8  -- Ranking functions
9  SELECT FullName, GPA, DeptID,
10     ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY GPA DESC) AS RowNum
11     ,
12     RANK() OVER (PARTITION BY DeptID ORDER BY GPA DESC) AS Rnk,
13     DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY GPA DESC) AS DRnk,
14     NTILE(4) OVER (ORDER BY GPA DESC) AS
15     Quartile,
16     PERCENT_RANK() OVER (ORDER BY GPA) AS
17     PctRank,
18     CUME_DIST() OVER (ORDER BY GPA) AS
19     CumDist
20 FROM Students;
21
22 -- Offset functions
23 SELECT OrderDate, Amount,
24     LAG(Amount, 1, 0) OVER (ORDER BY OrderDate) AS PrevAmount,
25     LEAD(Amount, 1, 0) OVER (ORDER BY OrderDate) AS NextAmount,
26     FIRST_VALUE(Amount) OVER (ORDER BY OrderDate
27     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
28     FirstInWindow,
29     LAST_VALUE(Amount) OVER (ORDER BY OrderDate
30     ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) AS LastInWindow
31 FROM Orders;
32
33 -- Aggregate window functions (per-row aggregates without collapsing rows
34 )
35 SELECT DeptID, FullName, GPA,
36     AVG(GPA) OVER (PARTITION BY DeptID) AS DeptAvgGPA,
37     COUNT(*) OVER (PARTITION BY DeptID) AS DeptSize
38 FROM Students;
39
40 -- Running total (typical finance/orders example)
41 SELECT OrderDate, Amount,
42     SUM(Amount) OVER (
43     ORDER BY OrderDate
44     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
45 ) AS RunningTotal
46 FROM Orders;

```

17. Indexes

Index Creation and Management

```

1  -- B-Tree index (default; best for range and equality queries)
2  CREATE INDEX idx_students_gpa ON Students (GPA);
3  CREATE INDEX idx_students_dept_gpa ON Students (DeptID, GPA); --
4  composite
5
6  -- Unique index
7  CREATE UNIQUE INDEX idx_students_email ON Students (Email);
8
9  -- Clustered index (T-SQL; one per table; determines physical order)
10 CREATE CLUSTERED INDEX idx_students_pk ON Students (StudentID);
11
12 -- Non-clustered with INCLUDE (covering index)
13 CREATE NONCLUSTERED INDEX idx_cover ON Students (DeptID)
14 INCLUDE (FullName, GPA); -- extra columns in leaf level; avoids

```

```

    table lookup
14
15 -- [PG]: All T-SQL index syntax is nearly identical in PostgreSQL
16 -- [PG]: CREATE INDEX CONCURRENTLY is available (non-locking)
17 CREATE INDEX CONCURRENTLY idx_students_gpa ON Students (GPA); -- PG only
18
19 -- Hash index (equality queries only)
20 CREATE INDEX idx_hash ON Students USING HASH (Email); -- PG syntax
21 -- T-SQL: hash indexes only available on memory-optimised tables
22
23 -- Drop index
24 DROP INDEX idx_students_gpa ON Students; -- T-SQL
25 DROP INDEX idx_students_gpa; -- PG (no table name needed)
26
27 -- View index usage
28 SELECT * FROM sys.indexes WHERE object_id = OBJECT_ID('Students'); -- T-SQL

```

18. Views

```

</> Views
1 -- Create a view
2 CREATE VIEW vw_HighPerformers AS
3 SELECT s.FullName, s.GPA, d.DeptName
4 FROM Students s JOIN Departments d ON s.DeptID = d.DeptID
5 WHERE s.GPA > 3.5;
6
7 -- Query a view exactly like a table
8 SELECT * FROM vw_HighPerformers ORDER BY GPA DESC;
9
10 -- Updatable view (simple views can accept INSERT/UPDATE/DELETE)
11 UPDATE vw_HighPerformers SET GPA = 3.9 WHERE FullName = 'Amina Hassan';
12
13 -- Alter a view (T-SQL)
14 ALTER VIEW vw_HighPerformers AS
15 SELECT s.FullName, s.GPA, s.Email, d.DeptName
16 FROM Students s JOIN Departments d ON s.DeptID = d.DeptID
17 WHERE s.GPA > 3.0;
18
19 -- Drop view
20 DROP VIEW IF EXISTS vw_HighPerformers;

```

19. Stored Procedures and Functions

```

</> Stored Procedures (T-SQL)
1 CREATE PROCEDURE GetStudentsByDept
2     @DeptID INT,
3     @MinGPA DECIMAL(3,2) = 0.0 -- default parameter
4 AS
5 BEGIN
6     SET NOCOUNT ON;

```

```

7      SELECT StudentID, FullName, GPA
8      FROM    Students
9      WHERE   DeptID = @DeptID AND GPA >= @MinGPA
10     ORDER  BY GPA DESC;
11 END;
12 GO
13
14 -- Execute
15 EXEC GetStudentsByDept @DeptID = 2, @MinGPA = 3.5;
16
17 -- Drop procedure
18 DROP PROCEDURE IF EXISTS GetStudentsByDept;

```

Functions (T-SQL)

```

1  -- Scalar function
2  CREATE FUNCTION dbo.GetLetterGrade (@GPA DECIMAL(3,2))
3  RETURNS NVARCHAR(2)
4  AS BEGIN
5      RETURN CASE
6          WHEN @GPA >= 3.7 THEN 'A'
7          WHEN @GPA >= 3.0 THEN 'B'
8          WHEN @GPA >= 2.0 THEN 'C'
9          ELSE 'D' END;
10 END;
11 GO
12
13 SELECT FullName, GPA, dbo.GetLetterGrade(GPA) AS Grade FROM Students;
14
15 -- Table-valued function
16 CREATE FUNCTION dbo.GetDeptStudents (@DeptID INT)
17 RETURNS TABLE AS RETURN
18     SELECT StudentID, FullName, GPA FROM Students WHERE DeptID = @DeptID;

```

PostgreSQL Functions (PL/pgSQL)

```

1  CREATE OR REPLACE FUNCTION get_dept_avg(p_dept_id INT)
2  RETURNS DECIMAL AS $$
3  DECLARE
4      v_avg DECIMAL;
5  BEGIN
6      SELECT AVG(GPA) INTO v_avg FROM Students WHERE DeptID = p_dept_id;
7      RETURN v_avg;
8  END;
9  $$ LANGUAGE plpgsql;
10
11 SELECT get_dept_avg(2);

```

20. Triggers

</> Triggers (T-SQL)

```

1  -- AFTER INSERT trigger: log new student enrollment
2  CREATE TRIGGER trg_AfterStudentInsert
3  ON Students
4  AFTER INSERT
5  AS BEGIN
6      SET NOCOUNT ON;
7      INSERT INTO AuditLog (Action, TableName, ActionTime)
8      SELECT 'INSERT', 'Students', GETDATE()
9      FROM inserted; -- virtual table with newly inserted rows
10 END;
11 GO
12
13 -- AFTER UPDATE trigger: prevent GPA from being lowered more than 1 point
14 CREATE TRIGGER trg_PreventGPADrop
15 ON Students
16 AFTER UPDATE
17 AS BEGIN
18     IF EXISTS (
19         SELECT 1 FROM inserted i JOIN deleted d ON i.StudentID = d.
20             StudentID
21         WHERE d.GPA - i.GPA > 1.0
22     )
23     BEGIN
24         ROLLBACK;
25         RAISERROR('GPA cannot be lowered by more than 1.0 point.', 16, 1)
26         ;
27     END;
28 END;
29 GO
30
31 -- [PG]: Triggers use a two-step approach (function + trigger)
32 CREATE OR REPLACE FUNCTION check_gpa_drop() RETURNS TRIGGER AS $$
33 BEGIN
34     IF OLD.GPA - NEW.GPA > 1.0 THEN
35         RAISE EXCEPTION 'GPA cannot drop by more than 1.0';
36     END IF;
37     RETURN NEW;
38 END;
39 $$ LANGUAGE plpgsql;
40
41 CREATE TRIGGER trg_gpa_check BEFORE UPDATE ON Students
42 FOR EACH ROW EXECUTE FUNCTION check_gpa_drop();

```

21. Transactions and Concurrency**</> Transactions (T-SQL)**

```

1  -- Basic transaction
2  BEGIN TRANSACTION;
3      UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
4      UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
5      IF @@ERROR <> 0
6          ROLLBACK;

```

```

7      ELSE
8          COMMIT;
9
10     -- TRY...CATCH transaction (recommended)
11     BEGIN TRY
12         BEGIN TRANSACTION;
13         UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
14         UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;
15         COMMIT TRANSACTION;
16     END TRY
17     BEGIN CATCH
18         ROLLBACK TRANSACTION;
19         THROW;
20     END CATCH;
21
22     -- SAVEPOINT
23     SAVEPOINT sp1;          -- PG syntax
24     ROLLBACK TO sp1;        -- PG syntax
25     SAVE TRANSACTION sp1;   -- T-SQL
26     ROLLBACK TRANSACTION sp1; -- T-SQL
27
28     -- Isolation levels (SET before BEGIN TRANSACTION)
29     SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED; -- dirty reads allowed
30     SET TRANSACTION ISOLATION LEVEL READ COMMITTED;   -- default in most DBs
31     SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  -- no non-repeatable
32     reads
33     SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;      -- strictest; no
34     phantoms
35     SET TRANSACTION ISOLATION LEVEL SNAPSHOT;          -- T-SQL only (MVCC)

```

22. Security — DCL

GRANT and REVOKE

```

1  -- Grant specific privileges
2  GRANT SELECT, INSERT ON Students TO ReadWriteRole;
3  GRANT EXECUTE ON GetStudentsByDept TO AppUser;
4  GRANT ALL PRIVILEGES ON Students TO AdminUser;      -- PG syntax
5  GRANT ALL ON Students TO AdminUser;                  -- T-SQL syntax
6
7  -- Grant with propagation
8  GRANT SELECT ON Students TO ReaderRole WITH GRANT OPTION;
9
10 -- Revoke privileges
11 REVOKE INSERT ON Students FROM ReadWriteRole;
12 REVOKE ALL ON Students FROM AppUser;
13
14 -- T-SQL: Create a login, user, and role
15 CREATE LOGIN AppLogin WITH PASSWORD = 'Str0ng!Pass';
16 CREATE USER AppUser FOR LOGIN AppLogin;
17 CREATE ROLE ReaderRole;
18 ALTER ROLE ReaderRole ADD MEMBER AppUser;
19 GRANT SELECT ON SCHEMA::dbo TO ReaderRole;

```

23. Vector Database: pgvector (PostgreSQL)

</> pgvector Setup and Queries

```

1  -- Install extension (once per database)
2  CREATE EXTENSION IF NOT EXISTS vector;
3
4  -- Add a vector column to the existing Documents table from the Reference
   Schema
5  -- (Documents.Content stores text; we add Documents.Embedding for
   embeddings)
6  ALTER TABLE Documents
7  ADD COLUMN Embedding vector(384); -- 384-dim for all-MiniLM-L6-v2 model
8
9  -- Insert a vector (from Python: pass as list)
10 INSERT INTO Documents (Content, Embedding)
11 VALUES ('Introduction to databases', '[0.12, -0.03, 0.45, ...]');
12
13 -- Cosine similarity search (most similar documents)
14 SELECT DocID, Content,
15        1 - (Embedding <=> '[0.10, -0.05, 0.48, ...]') AS CosineSim
16 FROM   Documents
17 ORDER BY Embedding <=> '[0.10, -0.05, 0.48, ...]'
18 LIMIT  5;
19
20 -- Operators: <=> cosine distance | <-> L2 (Euclidean) | <#> dot
   product
21
22 -- HNSW index for fast approximate nearest-neighbour search
23 CREATE INDEX idx_hnsw ON Documents
24 USING hnsw (Embedding vector_cosine_ops)
25 WITH (m = 16, ef_construction = 64);
26
27 -- IVFFlat index (alternative)
28 CREATE INDEX idx_ivfflat ON Documents
29 USING ivfflat (Embedding vector_cosine_ops) WITH (lists = 100);

```

24. Python Integration**</> Python + psycopg2 (PostgreSQL)**

```

1  import psycopg2
2
3  conn = psycopg2.connect(
4      host="localhost", database="z2004_db",
5      user="postgres", password="your_password"
6  )
7  cur = conn.cursor()
8
9  # Parameterised query (ALWAYS use parameterised; never f-strings for SQL)
10 cur.execute("SELECT * FROM Students WHERE DeptID = %s AND GPA > %s", (2,
11     3.5))
12 rows = cur.fetchall()
13 for row in rows:
14     print(row)
15
16 # Insert with parameters

```

```
16 cur.execute("""
17     INSERT INTO Students (FullName, Email, GPA, DeptID)
18     VALUES (%s, %s, %s, %s)
19 """, ("Amina Hassan", "amina@iitmz.ac.tz", 3.85, 2))
20 conn.commit()
21
22 cur.close()
23 conn.close()
```

Python + SQLAlchemy ORM

```
1 from sqlalchemy import create_engine, Column, Integer, String, Float
2 from sqlalchemy.orm import declarative_base, Session
3
4 engine = create_engine("postgresql://postgres:pass@localhost/z2004_db")
5 Base = declarative_base()
6
7 class Student(Base):
8     __tablename__ = "students"
9     StudentID = Column(Integer, primary_key=True)
10    FullName = Column(String(100), nullable=False)
11    GPA = Column(Float)
12
13 Base.metadata.create_all(engine)
14
15 with Session(engine) as session:
16     new_s = Student(FullName="Fatuma Ali", GPA=3.92)
17     session.add(new_s)
18     session.commit()
19
20     high_gpa = session.query(Student).filter(Student.GPA > 3.7).all()
21     for s in high_gpa:
22         print(s.FullName, s.GPA)
```

Python + pgvector (RAG Pipeline Snippet)

```
1 from sentence_transformers import SentenceTransformer
2 import psycopg2
3
4 model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions
5 conn = psycopg2.connect("postgresql://postgres:pass@localhost/z2004_db")
6 cur = conn.cursor()
7
8 # Store embedding
9 text = "IIT Madras Zanzibar offers B.S. in Data Science."
10 vec = model.encode(text).tolist()
11 cur.execute("INSERT INTO Documents (Content, Embedding) VALUES (%s, %s)",
12             (text, vec))
13 conn.commit()
14
15 # Query: find 3 most similar documents to a question
16 question = "What programmes does IIT Madras Zanzibar offer?"
17 q_vec = model.encode(question).tolist()
18 cur.execute("""
19     SELECT Content, 1 - (Embedding <=> %s) AS Similarity
20     FROM Documents
21     ORDER BY Embedding <=> %s
22     LIMIT 3
```

```
23 """ , (q_vec, q_vec))
24 results = cur.fetchall()
25 for content, sim in results:
26     print(f"[{sim:.3f}] {content}")
```